Interactive Web-Based 3D Gaussian Splatting Viewer with Idle-Time Image Refinement

Jheng-Ling Lee
National Taiwan University

Abstract

This report presents an interactive web-based viewer for 3D Gaussian Splatting (3DGS) scenes with an idle-time image refinement pipeline. The system is designed for a practical server-based demonstration setting, where camera control must remain responsive even though image enhancement is comparatively expensive. During active orbiting, panning, and zooming, the viewer displays raw 3DGS renderings to prioritize interaction latency. After the camera remains stationary for a short interval, the backend renders the current view at a higher-quality setting and applies Difix3D+-based image-space refinement. This separation allows the demo to preserve real-time exploration while improving selected still views after user interaction stops. Qualitative comparisons show that using more training images improves the underlying reconstruction, while refinement can suppress visible artifacts and smooth degraded regions in stationary views. Timing measurements further support the design choice by showing that refinement is substantially slower than raw rendering and is therefore better treated as a still-view enhancement rather than a continuous interactive rendering stage. The project repository is available at https://github.com/Leoxu3/3dgs-web.

1 Introduction

Novel view synthesis (NVS) aims to render images of a scene from camera viewpoints that were not directly observed during capture. For an interactive demo, this problem is not only about reconstruction quality, but also about whether the scene can be explored smoothly by a user. A practical viewer must therefore balance visual fidelity, camera responsiveness, and deployment constraints.

This project builds an interactive web-based viewer for 3D Gaussian Splatting (3DGS) scenes with an additional idle-time image refinement stage. Raw 3DGS rendering is used during camera movement to preserve responsive orbiting, panning, and zooming. When the camera becomes stationary, the current view can be refined by a backend module based on Difix3D+.

The main goal is not to propose a new reconstruction algorithm, but to combine existing 3DGS rendering and image refinement techniques into a demonstration system. The report focuses on the resulting design trade-off: real-time interaction is handled by the raw 3DGS renderer, while slower image enhancement is deferred until the user is inspecting a stable viewpoint. The following sections review the relevant NVS and refinement methods, describe the frontend and backend pipeline, and present qualitative demo results.

2 Related Work

2.1 Novel View Synthesis and Radiance Fields

Novel view synthesis aims to render plausible images of a scene from camera viewpoints that were not directly observed during capture. Earlier image-based rendering systems such as Local Light Field Fusion already emphasized a practical requirement that is also central to this project: a view synthesis method must support reliable capture and smooth visual exploration, not only isolated image quality [9]. Neural Radiance Fields (NeRF) later established a widely used radiance-field formulation for high-quality view synthesis from posed images [10]. This line of work made learned scene representations a standard reference point for NVS, while also making the cost of volumetric rendering an important deployment concern.

Subsequent radiance-field research can be viewed as a balance between visual fidelity, robustness, and efficiency. Methods such as Mip-NeRF 360 and Zip-NeRF address higher-quality reconstruction for challenging real-world scenes and scale changes [1, 2]. In parallel, explicit or factorized representations such as Plenoxels and TensoRF reduce optimization cost and model overhead while preserving the radiance-field rendering objective [5, 3]. These works are not used directly in the demo, but they motivate the broader design problem: an interactive system must decide which computation belongs in the real-time rendering loop and which computation can be deferred.

2.2 Interactive Rendering and 3D Gaussian Splatting

For an interactive viewer, the most relevant related work is the effort to move high-quality NVS toward real-time

rendering. SNeRG, PlenOctrees, Instant-NGP, and MobileNeRF show different ways to reduce the cost of rendering neural or learned radiance-field representations on practical hardware [6, 16, 11, 4]. The common lesson is more important here than any individual implementation: camera interaction should remain responsive, even if the representation or enhancement model is computationally expensive.

3D Gaussian Splatting (3DGS) is especially suitable for this project because it represents a scene using explicit Gaussian primitives and a rasterization-friendly rendering procedure, enabling real-time novel view synthesis for reconstructed scenes [8]. Tooling such as Nerfstudio further reflects the practical need to connect radiance-field methods to training pipelines, viewers, and user-facing demos [12]. In the implementation of this project, raw 3DGS rendering is performed through `gsplat`, an open-source Gaussian Splatting library that provides a PyTorch frontend and optimized CUDA rasterization kernels [15]. This library is important to the system design because it supplies the practical rendering backend used to convert loaded Gaussian scene parameters into browser-displayable frames.

At the same time, follow-up work on splatting has studied issues such as aliasing and geometric accuracy, which indicates that fast raw rendering does not remove all artifacts in difficult views [17, 7]. This motivates the project design: raw 3DGS rendering is used for interaction, while additional enhancement is reserved for stable viewpoints.

2.3 Image-Space Refinement

Image-space refinement methods address rendering artifacts after an image has already been produced by a reconstruction or rendering pipeline. This class of approaches is relevant when the underlying 3D representation is fast enough for interaction but still produces imperfect still views. Recent reconstruction work has also explored learned image priors for underconstrained observations; for example, ReconFusion uses diffusion priors to improve sparse-view 3D reconstruction [14]. Such methods support the general idea that image priors can help with regions where the captured views provide limited evidence, although they may also introduce consistency and hallucination risks.

DIFIX3D+ is particularly relevant because it targets artifacts in rendered novel views from both NeRF and 3DGS representations, and it can also be used as an inference-time enhancer for rendered views [13]. This inference-time use case matches the demo system more closely than methods that require re-optimizing the entire 3D scene. This project adopts image-space refinement as a post-rendering stage rather than as a method for re-optimizing the 3D Gaussian scene.

3 Method

3.1 System Overview

The system consists of a browser-based frontend and a Python backend, as shown in Figure 1. The frontend provides camera interaction and controls when rendering or refinement should be requested. The backend is responsible for loading the 3DGS scene, producing raw rendered views, and applying image-space refinement when the camera is idle.

The rendering pipeline follows two operating modes. During active interaction, the viewer displays raw 3DGS frames to maintain responsive camera control. During idle periods, the system renders the stable view at a higher resolution and then produces a refined still image for inspection.

3.2 Interactive Rendering

The frontend is responsible for camera interaction and for displaying rendered frames returned by the backend. Camera controls allow the user to orbit, pan, and zoom around the reconstructed scene. When the camera changes, the frontend sends the current viewpoint and display size to the backend and requests an interactive raw render.

The backend renderer loads the selected `.ply` scene, extracts Gaussian attributes such as positions, scales, rotations, opacity, and color features when available, and invokes the `gsplat` rasterization API to render the requested camera view. During camera movement, the render resolution is reduced to favor lower latency and smoother interaction. The rendered image is generated on the backend and then displayed by the browser.

Since continuous refinement would introduce excessive latency, the viewer prioritizes raw `gsplat`-based 3DGS rendering while the camera is moving. The implementation also keeps a fallback renderer for cases where CUDA, the `gsplat` package, or a compatible Gaussian `.ply` file is unavailable, which makes the web demo easier to run on server environments with different GPU configurations.

3.3 Idle-Time Refinement

After camera motion stops, the frontend waits for a short idle interval before triggering refinement. This delay prevents the system from sending unnecessary requests while the user is still adjusting the viewpoint. Once the view is considered stable, the frontend asks the backend to render the same view using an idle-quality setting. This idle render uses a higher resolution than the moving-camera render, and both the raw idle view and the refined output are produced from this higher-resolution backend rendering path.

The idle pipeline is performed on the backend rather than from a client-side screenshot. The backend first renders the current view with the same 3DGS renderer used during interaction, then passes that rendered image

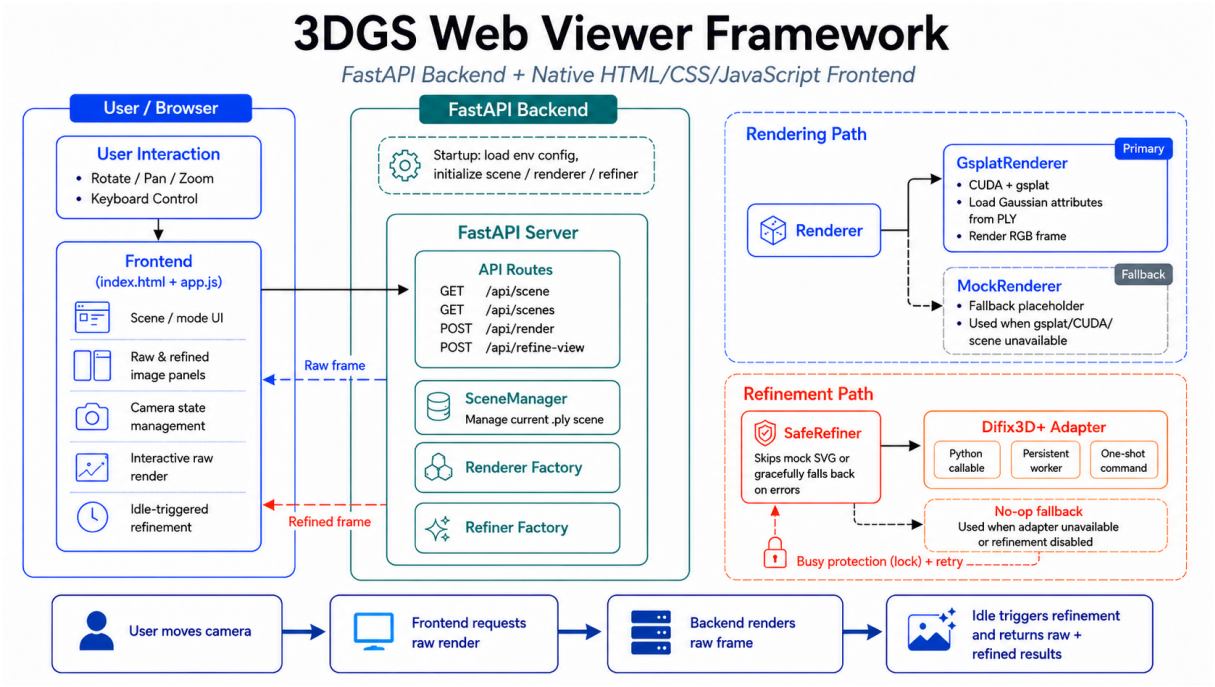

Figure 1: System overview of the web-based 3DGS viewer. The browser controls the camera and display state, while the backend performs raw 3DGS rendering and idle-time Difix3D+-based refinement.

to a Difix3D+ adapter for image-space refinement [13]. The browser receives both the raw idle render and the refined image, allowing the interface to compare or switch between them.

Refinement requests are guarded by a busy retry lock that acts as a single-request gate for the expensive refinement pipeline. This lock is used as a back-pressure mechanism rather than a blocking queue: if another refinement is already running, the backend immediately returns a busy result instead of waiting for the previous frame to finish. The frontend then retries after a short delay only when the camera state is still current; otherwise, the obsolete request is discarded and a later idle request uses the latest camera pose. This policy prevents stale frames from accumulating and avoids repeatedly overloading the GPU or persistent refinement worker.

The returned refined image is displayed as an enhanced still view for the current camera pose. If the user starts moving the camera again before the request finishes, the frontend invalidates the pending refinement and returns to raw 3DGS rendering so that camera interaction remains responsive. The implementation also warms up the refiner before the first visible idle refinement request when possible, reducing first-request model-loading overhead.

4 Experiments

The project is evaluated through qualitative comparison and measured rendering latency from the demo system. The evaluation considers three questions. First, how does the number of training images affect the quality of the reconstructed 3DGS scene? Second, how does image-space refinement change the appearance of a stationary rendered view? Third, how much latency is introduced by the idle-quality rendering and refinement stages? The visual results are shown as screenshots from the web-based viewer, while the timing results report average render time under the tested demo configuration. All timing experiments were conducted on a server equipped with an NVIDIA RTX A6000 GPU.

Figure 2 compares raw 3DGS renderings produced from different numbers of input photographs. When only 30 images are used, the scene contains more visible degradation and less stable detail. In contrast, the reconstruction trained with 300 images better preserves the global scene structure and produces sharper local appearance. This result is consistent with the role of multi-view coverage in 3D reconstruction: more observations reduce ambiguity and provide stronger constraints for geometry and appearance optimization.

Figure 3 compares the raw rendering with the refined output at a stationary camera pose. The refined image is visually smoother and contains fewer apparent rendering errors and degradation artifacts. This behavior is useful for the intended demo setting: while the user moves the camera, the raw 3DGS output preserves interactive response; after the camera becomes idle, refinement can improve the still image that the user is inspecting.

However, Figure 2 also reveals an important trade-off. The refinement stage can smooth degraded regions, but it does not always preserve high-frequency content faithfully.

(a) Raw 3DGS render trained with 30 images.

(b) Raw 3DGS render trained with 300 images.

(c) Refined output from the 30-image reconstruction.

(d) Refined output from the 300-image reconstruction.

Figure 2: Effect of training image count and refinement on the center view. With more input photographs from different viewpoints, the reconstructed scene receives denser multi-view evidence and produces a clearer and more accurate view. Refinement smooths visible degradation, but it does not fully replace the need for a well-constrained 3D reconstruction.

Table 1: Average render time measured in the demo system.

Camera State	Output	Average Time
Moving	Raw 3DGS render	71.68 ms
Idle	Raw 3DGS render	198.23 ms
Idle	Refined output	1006.76 ms

In regions containing text or other fine structures, the refined image may appear less sharp than the corresponding raw render. Therefore, refinement should be interpreted as a perceptual image enhancement step rather than a guaranteed correction of the underlying 3D scene.

Table 1 reports the average render time measured for the three display states used by the viewer. The moving-camera raw render is the fastest state, averaging 71.68 ms per rendered view, because it prioritizes responsiveness during interaction. The idle raw render averages 198.23 ms because it uses the higher-quality idle rendering path before refinement. The refined output averages 1006.76 ms, which confirms that Difix3D+-based enhancement is substantially slower than raw rendering and should remain outside the continuous camera-control loop. In the tested configuration, the refined output is approximately 5 times slower than the idle raw render and 14 times slower than the moving raw render.

5 Limitations and Future Work

The system design exposes several important limitations. First, Difix3D-based refinement introduces latency and is therefore unsuitable for true real-time frame-by-frame interaction. Second, image-space refinement does not modify the underlying 3D Gaussian representation, so improvements are limited to the current rendered image. Third, independently refined views may have limited temporal or multi-view consistency. Finally, the demo depends on backend GPU availability, model checkpoint loading time, and server response stability.

Future work may include stronger request scheduling, broader latency profiling across scenes and hardware, view-consistent refinement, and tighter integration between the refinement module and the 3D scene representation.

6 Conclusion

This project builds a practical web-based 3DGS demonstration system with an idle-time refinement pipeline. By separating interactive rendering from slower image enhancement, the system supports responsive camera control while still allowing selected views to be improved after interaction stops. The measured render times further support this separation: raw rendering remains more suitable

(a) Raw 3DGS render at the evaluated pose.

(b) Refined output at the same pose.

Figure 3: Qualitative comparison between raw 3DGS rendering and image-space refinement at a stationary camera pose. Refinement makes the overall scene smoother and suppresses many local artifacts.

for interaction, while refined output is better treated as a still-view enhancement.

References

[1] Jonathan T. Barron, Ben Mildenhall, Dor Verbin, Pratul P. Srinivasan, and Peter Hedman. Mip-NeRF 360: Unbounded anti-aliased neural radiance fields. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 5470–5479, June 2022.

[2] Jonathan T. Barron, Ben Mildenhall, Dor Verbin, Pratul P. Srinivasan, and Peter Hedman. Zip-NeRF: Anti-aliased grid-based neural radiance fields. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 19697–19705, October 2023.

[3] Anpei Chen, Zexiang Xu, Andreas Geiger, Jingyi Yu, and Hao Su. TensoRF: Tensorial radiance fields. In *Computer Vision – ECCV 2022*, pages 333–350. Springer, 2022.

[4] Zhiqin Chen, Thomas Funkhouser, Peter Hedman, and Andrea Tagliasacchi. MobileNeRF: Exploiting the polygon rasterization pipeline for efficient neural field rendering on mobile architectures. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 16569–16578, June 2023.

[5] Sara Fridovich-Keil, Alex Yu, Matthew Tancik, Qinhong Chen, Benjamin Recht, and Angjoo Kanazawa. Plenoxels: Radiance fields without neural networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 5501–5510, June 2022.

[6] Peter Hedman, Pratul P. Srinivasan, Ben Mildenhall, Jonathan T. Barron, and Paul Debevec. Baking neural radiance fields for real-time view synthesis. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 5875–5884, October 2021.

[7] Binbin Huang, Zehao Yu, Anpei Chen, Andreas Geiger, and Shenghua Gao. 2D gaussian splatting for geometrically accurate radiance fields. In *ACM SIGGRAPH 2024 Conference Papers*, pages 1–11. ACM, 2024.

[8] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3D gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics*, 42(4), 2023.

[9] Ben Mildenhall, Pratul P. Srinivasan, Rodrigo Ortiz-Cayon, Nima Khademi Kalantari, Ravi Ramamoorthi, Ren Ng, and Abhishek Kar. Local light field fusion: Practical view synthesis with prescriptive sampling guidelines. *ACM Transactions on Graphics*, 38(4):29:1–29:14, 2019.

[10] Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. NeRF: Representing scenes as neural radiance fields for view synthesis. In *Computer Vision – ECCV 2020*, pages 405–421. Springer, 2020.

[11] Thomas Müller, Alex Evans, Christoph Schied, and Alexander Keller. Instant neural graphics primitives with a multiresolution hash encoding. *ACM Transactions on Graphics*, 41(4):102:1–102:15, 2022.

[12] Matthew Tancik, Ethan Weber, Evonne Ng, Ruilong Li, Brent Yi, Justin Kerr, Terrance Wang, Alexander Kristoffersen, Jake Austin, Kamyar Salahi, Abhik Ahuja, David McAllister, and Angjoo Kanazawa. Nerfstudio: A modular framework for neural radiance field development. In *ACM SIGGRAPH 2023 Conference Proceedings*, SIGGRAPH '23, 2023.

[13] Jay Zhangjie Wu, Yuxuan Zhang, Haithem Turki, Xuanchi Ren, Jun Gao, Mike Zheng Shou, Sanja Fidler, Zan Gojcic, and Huan Ling. DIFIX3D+: Improving

3D reconstructions with single-step diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 26024–26035, June 2025.

[14] Rundi Wu, Ben Mildenhall, Philipp Henzler, Keunhong Park, Ruiqi Gao, Daniel Watson, Pratul P. Srinivasan, Dor Verbin, Jonathan T. Barron, Ben Poole, and Aleksander Holynski. ReconFusion: 3D reconstruction with diffusion priors. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 21551–21561, June 2024.

[15] Vickie Ye, Ruilong Li, Justin Kerr, Matias Turkulainen, Brent Yi, Zhuoyang Pan, Otto Seiskari, Jianbo Ye, Jeffrey Hu, Matthew Tancik, and Angjoo Kanazawa. gsplat: An open-source library for gaussian splatting. *Journal of Machine Learning Research*, 26(34):1–17, 2025.

[16] Alex Yu, Ruilong Li, Matthew Tancik, Hao Li, Ren Ng, and Angjoo Kanazawa. PlenOctrees for real-time rendering of neural radiance fields. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 5752–5761, October 2021.

[17] Zehao Yu, Anpei Chen, Binbin Huang, Torsten Sattler, and Andreas Geiger. Mip-splatting: Alias-free 3D gaussian splatting. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 19447–19456, June 2024.